

Name : Bhure Sangmeshwar Dattatray

Roll No : 07

Assignment 1: Parallel BFS and DFS using OpenMP
Undirected Graph representation using Adjacency List

```
#include <iostream>
#include <vector>
#include <queue>
#include <stack>
#include <omp.h>
#include <chrono>
using namespace std;

class Graph
{ public:
    int V;
    vector<vector<int>>> adj;

    Graph(int v) : V(v), adj(v) {}

    void addEdge(int u, int v) {
        adj[u].push_back(v);
        adj[v].push_back(u); // undirected
    }

    // ——— Sequential BFS —————
    void BFS_Sequential(int start) {
        vector<bool> visited(V, false);
        queue<int> q;
        visited[start] = true;
        q.push(start);

        cout << "\n[Sequential BFS] Visit order: ";
        while (!q.empty()) {
            int node = q.front(); q.pop();
            cout << node << " ";
            for (int nbr : adj[node]) {
                if (!visited[nbr]) {
                    visited[nbr] = true;
                    q.push(nbr);
                }
            }
        }
    }
};
```

```

    }
}
cout << endl;
}

// — Parallel BFS (level-synchronised)
void BFS_Parallel(int start) {
    vector<bool> visited(V, false);
    vector<int> current_level, next_level;

    visited[start] = true;
    current_level.push_back(start);

    cout << "\n[Parallel BFS] Visit order: ";
    while (!current_level.empty()) {
        // Print current level nodes
        for (int node : current_level) cout << node << " ";

        next_level.clear();

        // Process all nodes in current level in parallel
        #pragma omp parallel
        {
            vector<int> local_next;
            #pragma omp for nowait schedule(dynamic)
            for (int i = 0; i < (int)current_level.size(); i++) {
                int node = current_level[i];
                for (int nbr : adj[node]) {
                    bool already;
                    #pragma omp critical
                    {
                        already = visited[nbr];
                        if (!already) visited[nbr] = true;
                    }
                    if (!already) local_next.push_back(nbr);
                }
            }
            #pragma omp critical
            next_level.insert(next_level.end(), local_next.begin(), local_next.end());
        }
        current_level = next_level;
    }
    cout << endl;
}

```

```

// ——— Sequential DFS ———
void DFS_Sequential(int start) {
    vector<bool> visited(V, false);
    stack<int> st;
    st.push(start);

    cout << "\n[Sequential DFS] Visit order: ";
    while (!st.empty()) {
        int node = st.top(); st.pop();
        if (!visited[node])
            { visited[node] = true;
              cout << node << " ";
              for (int i = (int)adj[node].size() - 1; i >= 0; i--)
                  if (!visited[adj[node][i]])
                      st.push(adj[node][i]);
            }
    }
    cout << endl;
}

// ——— Parallel DFS (parallel neighbour exploration) ———
void DFS_Parallel_Helper(int node, vector<bool>& visited) {
    #pragma omp critical
    { visited[node] = true; }
    cout << node << " ";

    #pragma omp parallel for schedule(dynamic)
    for (int i = 0; i < (int)adj[node].size(); i++) {
        int nbr = adj[node][i];
        bool v;
        #pragma omp critical
        { v = visited[nbr]; }
        if (!v) DFS_Parallel_Helper(nbr, visited);
    }
}

void DFS_Parallel(int start)
{ vector<bool> visited(V,
false);
  cout << "\n[Parallel DFS] Visit order: ";
  DFS_Parallel_Helper(start, visited);
  cout << endl;
}
};

```

```

int main() {
    // Build a sample undirected graph with 8 nodes
    // 0 - 1 - 3 - 7
    // | |
    // 2 4 - 5
    //   |
    //   6
    Graph g(8);
    g.addEdge(0, 1); g.addEdge(0, 2);
    g.addEdge(1, 3); g.addEdge(1, 4);
    g.addEdge(3, 7); g.addEdge(4, 5);
    g.addEdge(4, 6);

    cout << "=====" << endl;
    cout << " Parallel BFS & DFS using OpenMP  " << endl;
    cout << " Graph: 8 nodes, start node = 0  " << endl;
    cout << "=====" << endl;

    // — BFS timing
    _____ auto t1 =
    chrono::high_resolution_clock::now();
    g.BFS_Sequential(0);
    auto t2 = chrono::high_resolution_clock::now();
    cout << " Time (Sequential BFS): "
        << chrono::duration_cast<chrono::microseconds>(t2-t1).count() << " μs\n";

    t1 = chrono::high_resolution_clock::now();
    g.BFS_Parallel(0);
    t2 = chrono::high_resolution_clock::now();
    cout << " Time (Parallel BFS): "
        << chrono::duration_cast<chrono::microseconds>(t2-t1).count() << " μs\n";

    // — DFS timing
    _____ t1 =
    chrono::high_resolution_clock::now();
    g.DFS_Sequential(0);
    t2 = chrono::high_resolution_clock::now();
    cout << " Time (Sequential DFS): "
        << chrono::duration_cast<chrono::microseconds>(t2-t1).count() << " μs\n";

    t1 = chrono::high_resolution_clock::now();
    g.DFS_Parallel(0);
    t2 = chrono::high_resolution_clock::now();
    cout << " Time (Parallel DFS): "
        << chrono::duration_cast<chrono::microseconds>(t2-t1).count() << " μs\n";
}

```

```
    return 0;  
}
```

// — Compile & Run —
// g++ -fopenmp -O2 -o a1 assignment1_bfs_dfs.cpp && ./a1

Output:

```
vboxuser@ubuntu: ~/LP_V_Code/HPC  
g++ (Ubuntu 13.3.0-6ubuntu2~24.04.1) 13.3.0  
Copyright (C) 2023 Free Software Foundation, Inc.  
This is free software; see the source for copying conditions.  There is NO  
warranty; not even for MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.  
  
vboxuser@ubuntu:~/LP_V_Code/HPC$ g++ -fopenmp -O2 -o a1 assignment1_bfs_dfs.cpp  
vboxuser@ubuntu:~/LP_V_Code/HPC$ ./a1  
=====  
Parallel BFS & DFS using OpenMP  
Graph: 8 nodes, start node = 0  
=====  
  
[Sequential BFS] Visit order: 0 1 2 3 4 7 5 6  
Time (Sequential BFS): 52 µs  
  
[Parallel BFS] Visit order: 0 1 2 3 4 5 6 7  
Time (Parallel BFS): 1649 µs  
  
[Sequential DFS] Visit order: 0 1 3 7 4 5 6 2  
Time (Sequential DFS): 4 µs  
  
[Parallel DFS] Visit order: 0 2 1 3 7 4 5 6  
Time (Parallel DFS): 22 µs  
vboxuser@ubuntu:~/LP_V_Code/HPC$
```

Name : Bhure Sangmeshwar Dattatray

Roll No : 07

Assignment 2: Parallel Bubble Sort & Merge Sort (OpenMP)

Measures and compares sequential vs parallel performance

```
#include <iostream>
#include <vector>
#include <algorithm>
#include <omp.h>
#include <chrono>
#include <random>
using namespace std;

//


---


// BUBBLE SORT
//


---



void bubbleSort_Sequential(vector<int>& arr)
{ int n = arr.size();
  for (int i = 0; i < n - 1; i++)
    for (int j = 0; j < n - i - 1; j++)
      if (arr[j] > arr[j+1])
        swap(arr[j], arr[j+1]);
}

// Odd-Even Transposition Sort (parallelisable bubble sort variant)
void bubbleSort_Parallel(vector<int>& arr) {
  int n = arr.size();
  for (int phase = 0; phase < n; phase++)
    { if (phase % 2 == 0) {
      // Even phase: compare (0,1), (2,3), (4,5) ...
      #pragma omp parallel for schedule(static)
      for (int i = 0; i < n - 1; i += 2) {
        if (arr[i] > arr[i+1])
          swap(arr[i], arr[i+1]);
      }
    } else {
      // Odd phase: compare (1,2), (3,4), (5,6) ...
      #pragma omp parallel for schedule(static)
```

```

        for (int i = 1; i < n - 1; i += 2)
            { if (arr[i] > arr[i+1])
              swap(arr[i], arr[i+1]);
            }
        }
    }
}

```

```
//
```

```
// MERGE SORT
```

```
//
```

```

void merge(vector<int>& arr, int l, int m, int r)
{ vector<int> left(arr.begin()+l, arr.begin()+m+1);
  vector<int> right(arr.begin()+m+1, arr.begin()+r+1);
  int i = 0, j = 0, k = l;
  while (i < (int)left.size() && j < (int)right.size()) arr[k++] = (left[i] <= right[j]) ? left[i++] : right[j++];
  while (i < (int)left.size()) arr[k++] = left[i++];
  while (j < (int)right.size()) arr[k++] = right[j++];
}

```

```

void mergeSort_Sequential(vector<int>& arr, int l, int r) {
    if (l >= r) return;
    int m = (l + r) / 2;
    mergeSort_Sequential(arr, l, m);
    mergeSort_Sequential(arr, m+1, r);
    merge(arr, l, m, r);
}

```

```

void mergeSort_Parallel(vector<int>& arr, int l, int r, int depth = 0)
{ if (l >= r) return;
  int m = (l + r) / 2;

  if (depth < 4) { // spawn threads up to depth 4
      #pragma omp parallel sections
      {
          #pragma omp section
          mergeSort_Parallel(arr, l, m, depth+1);
          #pragma omp section
          mergeSort_Parallel(arr, m+1, r, depth+1);
      }
  }
}

```

```
    } else {  
        mergeSort_Sequential(arr, l, m);  
        mergeSort_Sequential(arr, m+1, r);  
    }  
    merge(arr, l, m, r);  
}
```

```
//
```

```
// UTILITY
```

```
//
```

```
vector<int> generateRandom(int n)  
{ vector<int> v(n);  
  mt19937 rng(42);  
  uniform_int_distribution<int> dist(1, 100000);  
  for (auto& x : v) x = dist(rng);  
  return v;  
}
```

```
bool isSorted(const vector<int>& v)  
{ for (int i = 0; i + 1 < (int)v.size(); i++)  
  if (v[i] > v[i+1]) return false;  
  return true;  
}
```

```
void printArr(const vector<int>& v, int limit = 10)  
{ for (int i = 0; i < min((int)v.size(), limit); i++)  
  cout << v[i] << " ";  
  if ((int)v.size() > limit) cout << "...";  
  cout << endl;  
}
```

```
//
```

```
// MAIN
```

```
//
```

```
int main() {  
    const int BUBBLE_N = 5000;  
    const int MERGE_N = 500000;
```



```

cout << "=====" << endl;
cout << " Parallel Sorting Algorithms using OpenMP " << endl;
cout << "=====" << endl;

// — Bubble Sort ————— cout
<< "\n--- Bubble Sort (n = " << BUBBLE_N << ") ---" << endl;

auto orig = generateRandom(BUBBLE_N);

auto arr1 = orig;
auto s1 = chrono::high_resolution_clock::now();
bubbleSort_Sequential(arr1);
auto e1 = chrono::high_resolution_clock::now();
double t_bubble_seq = chrono::duration<double, milli>(e1-s1).count();
cout << "Sequential sorted: " << (isSorted(arr1) ? "YES" : "NO")
    << " | Time: " << t_bubble_seq << " ms" << endl;

auto arr2 = orig;
auto s2 = chrono::high_resolution_clock::now();
bubbleSort_Parallel(arr2);
auto e2 = chrono::high_resolution_clock::now();
double t_bubble_par = chrono::duration<double, milli>(e2-s2).count();
cout << "Parallel sorted: " << (isSorted(arr2) ? "YES" : "NO")
    << " | Time: " << t_bubble_par << " ms" << endl;
cout << "Speedup (Bubble): " << t_bubble_seq / t_bubble_par << "x" << endl;

// — Merge Sort —————
Merge Sort (n = " << MERGE_N << ") ---" << endl;

auto big = generateRandom(MERGE_N);

auto arr3 = big;
s1 = chrono::high_resolution_clock::now();
mergeSort_Sequential(arr3, 0, (int)arr3.size()-1);
e1 = chrono::high_resolution_clock::now();
double t_merge_seq = chrono::duration<double, milli>(e1-s1).count();
cout << "Sequential sorted: " << (isSorted(arr3) ? "YES" : "NO")
    << " | Time: " << t_merge_seq << " ms" << endl;

auto arr4 = big;
s2 = chrono::high_resolution_clock::now();
mergeSort_Parallel(arr4, 0, (int)arr4.size()-1);
e2 = chrono::high_resolution_clock::now();
double t_merge_par = chrono::duration<double, milli>(e2-s2).count();

```

```

cout << "Parallel sorted: " << (isSorted(arr4) ? "YES" : "NO")
    << " | Time: " << t_merge_par << " ms" << endl;
cout << "Speedup (Merge): " << t_merge_seq / t_merge_par << "x" << endl;

cout << "\nFirst 10 elements of sorted array: ";
printArr(arr4);

return 0;
}

```

// — Compile & Run —

```
// g++ -fopenmp -O2 -o a2 assignment2_sorting.cpp && ./a2
```

Output:

```

vboxuser@ubuntu: ~/LP V_Code/HPC
vboxuser@ubuntu:~/LP V_Code/HPC$ g++ -fopenmp -O2 -o a2 assignment2_sorting.cpp
vboxuser@ubuntu:~/LP V_Code/HPC$ ./a2
=====
Parallel Sorting Algorithms using OpenMP
=====

--- Bubble Sort (n = 5000) ---
Sequential sorted: YES | Time: 45.1287 ms
Parallel sorted: YES | Time: 42.7173 ms
Speedup (Bubble): 1.05645x

--- Merge Sort (n = 500000) ---
Sequential sorted: YES | Time: 67.2157 ms
Parallel sorted: YES | Time: 63.7905 ms
Speedup (Merge): 1.05369x

First 10 elements of sorted array: 1 1 1 1 1 1 1 1 2 2 ...
vboxuser@ubuntu:~/LP V_Code/HPC$

```

Name : Bhure Sangmeshwar Dattatray

Roll No : 07

Assignment 3: Parallel Reduction (Min, Max, Sum, Average)

Using OpenMP reduction clause

```
#include <iostream>
#include <vector>
#include <climits>
#include <cfloat>
#include <omp.h>
#include <chrono>
#include <random>
using namespace std;
```

```
//
```

```
// SEQUENTIAL implementations
```

```
//
```

```
int seq_min(const vector<int>& arr) {
    int mn = INT_MAX;
    for (int x : arr) mn = min(mn, x);
    return mn;
}
int seq_max(const vector<int>& arr)
{ int mx = INT_MIN;
  for (int x : arr) mx = max(mx, x);
  return mx;
}
long long seq_sum(const vector<int>& arr)
{ long long s = 0;
  for (int x : arr) s += x;
  return s;
}
double seq_avg(const vector<int>& arr)
{ return (double)seq_sum(arr) / arr.size();
}
```

```
//
```

```
// PARALLEL implementations using OpenMP reduction
//
```

```
int par_min(const vector<int>& arr)
{ int mn = INT_MAX;
  #pragma omp parallel for reduction(min:mn) schedule(static)
  for (int i = 0; i < (int)arr.size(); i++)
    mn = min(mn, arr[i]);
  return mn;
}
```

```
int par_max(const vector<int>& arr)
{ int mx = INT_MIN;
  #pragma omp parallel for reduction(max:mx) schedule(static)
  for (int i = 0; i < (int)arr.size(); i++)
    mx = max(mx, arr[i]);
  return mx;
}
```

```
long long par_sum(const vector<int>& arr)
{ long long s = 0;
  #pragma omp parallel for reduction(+:s) schedule(static)
  for (int i = 0; i < (int)arr.size(); i++)
    s += arr[i];
  return s;
}
```

```
double par_avg(const vector<int>& arr)
{ return (double)par_sum(arr) / arr.size();
}
```

```
//
```

```
// HELPER
//
```

```
template<typename Func>
double timelt(Func f) {
  auto s = chrono::high_resolution_clock::now();
  f();
  auto e = chrono::high_resolution_clock::now();
  return chrono::duration<double, milli>(e-s).count();
}
```

```

}

//

```

```

// MAIN
//

```

```

int main() {
    const int N = 10'000'000;

    // Generate random data
    vector<int> arr(N);
    mt19937 rng(99);
    uniform_int_distribution<int> dist(1, 1000000);
    for (auto& x : arr) x = dist(rng);

    cout << "===== " << endl;
    cout << " Parallel Reduction (Min/Max/Sum/Avg) via OpenMP " << endl;
    cout << " Array size: " << N << " elements " << endl;
    cout << "===== " << endl;
    cout << "Available threads: " << omp_get_max_threads() << endl;

    //                                     — —                                     M I N
    _____ int s_min,
    p_min;
    double t_s = timelt([&]{ s_min = seq_min(arr); });
    double t_p = timelt([&]{ p_min = par_min(arr); });
    cout << "\n--- MIN ---" << endl;
    cout << " Sequential: " << s_min << " Time: " << t_s << " ms" << endl;
    cout << " Parallel: " << p_min << " Time: " << t_p << " ms"
        << " Speedup: " << t_s/t_p << "x" << endl;
    cout << " Match: " << (s_min == p_min ? "YES" : "NO") << endl;

    //                                     — —                                     M A X
    _____ int
    s_max, p_max;
    t_s = timelt([&]{ s_max = seq_max(arr); });
    t_p = timelt([&]{ p_max = par_max(arr); });
    cout << "\n--- MAX ---" << endl;
    cout << " Sequential: " << s_max << " Time: " << t_s << " ms" << endl;
    cout << " Parallel: " << p_max << " Time: " << t_p << " ms"
        << " Speedup: " << t_s/t_p << "x" << endl;
    cout << " Match: " << (s_max == p_max ? "YES" : "NO") << endl;

    // — SUM —

```

```

long long s_sum, p_sum;
t_s = timelt([&]{ s_sum = seq_sum(arr); });
t_p = timelt([&]{ p_sum = par_sum(arr); });
cout << "\n--- SUM ---" << endl;
cout << " Sequential: " << s_sum << " Time: " << t_s << " ms" << endl;
cout << " Parallel: " << p_sum << " Time: " << t_p << " ms"
    << " Speedup: " << t_s/t_p << "x" << endl;
cout << " Match: " << (s_sum == p_sum ? "YES" : "NO") << endl;

// — AVERAGE —————
double s_avg, p_avg;
t_s = timelt([&]{ s_avg = seq_avg(arr); });
t_p = timelt([&]{ p_avg = par_avg(arr); });
cout << "\n--- AVERAGE ---" << endl;
cout << " Sequential: " << s_avg << " Time: " << t_s << " ms" << endl;
cout << " Parallel: " << p_avg << " Time: " << t_p << " ms"
    << " Speedup: " << t_s/t_p << "x" << endl;
cout << " Match: " << (abs(s_avg - p_avg) < 1e-6 ? "YES" : "NO") << endl;

cout << "\n===== " << endl;

return 0;
}

// — Compile & Run —————
// g++ -fopenmp -O2 -o a3 assignment3_reduction.cpp && ./a3

```

Output:

```

vboxuser@ubuntu:~/LP V_Code/HPC$ g++ -fopenmp -O2 -o a3 assignment3_reduction.cpp
vboxuser@ubuntu:~/LP V_Code/HPC$ ./a3
=====
Parallel Reduction (Min/Max/Sum/Avg) via OpenMP
Array size: 10000000 elements
=====
Available threads: 5

--- MIN ---
Sequential: 1 Time: 10.0472 ms
Parallel: 1 Time: 10.713 ms Speedup: 0.937852x
Match: YES

--- MAX ---
Sequential: 1000000 Time: 28.0437 ms
Parallel: 1000000 Time: 5.01466 ms Speedup: 5.59235x
Match: YES

--- SUM ---
Sequential: 5000093352437 Time: 6.44629 ms
Parallel: 5000093352437 Time: 5.22957 ms Speedup: 1.23266x
Match: YES

--- AVERAGE ---
Sequential: 500009 Time: 7.86292 ms
Parallel: 500009 Time: 4.60576 ms Speedup: 1.70719x
Match: YES

=====
vboxuser@ubuntu:~/LP V_Code/HPC$

```

Name : Bhure Sangmeshwar Dattatray

Roll No : 07

Assignment 3

[1]:

```
nvdiia-smi
```

Mon Apr 13 08:01:48 2026

```
+-----+
+-----+
| NVIDIA-SMI 580.82.07                  Driver Version: 580.82.07          CUDA Version:
13.0   |
+-----+-----+-----+
+-----+
| GPU  Name                Persistence-M | Bus-Id                Disp.A | Volatile
Uncorr. ECC |
| Fan   Temp   Perf         Pwr:Usage/Cap |      Memory-Usage | GPU-Util
Compute M. |
| MIG M. |
+=====+-----+-----+-----+
=====|
|    0   Tesla  T4               Off  |    00000000:00:04:0  Off  |
0  |
| N/A    36C    P8              9W /   70W |      0MiB /   15360MiB |      0%
Default |
|                                     |
N/A |
+-----+-----+-----+
+-----+

+-----+
+-----+
| Processes:
|
| GPU   GI    CI          PID    Type    Process name
GPU Memory |
|      ID    ID
Usage      |
+=====+
=====|
| No running processes found
|
```

+-----+
-----+

[2]: `from google.colab import files`
`uploaded = files.upload()`

<IPython.core.display.HTML object>

Saving assignment4_cuda.cu to assignment4_cuda.cu

[3]: `!nvcc assignment4_cuda.cu -o a4`

nvcc warning : Support for offline compilation for architectures prior to '<compute/sm/lto>_75' will be removed in a future release (Use -Wno-deprecated-gpu-targets to suppress warning).

[4]: `!./a4`

```
=====
Matrix Multiplication      (512x512)
=====
CPU time:                  737.59  ms
GPU Naive time:            122.18 ms Speedup: 6.04x GPU
Tiled time:                0.54 ms  Speedup: 1374.55x
Naive Verify: PASS        Tiled Verify: PASS
```


Name : Bhure Sangmeshwar Dattatray

Roll No : 07

Assignment 5: HPC Application for AI/ML Domain

Parallel Neural Network (Multi-Layer Perceptron) Training using OpenMP for forward pass, backprop & weight updates

Dataset: XOR problem + synthetic classification

```
#include <iostream>
#include <vector>
#include <cmath>
#include <random>
#include <omp.h>
#include <chrono>
#include <iomanip>
using namespace std;

//


---


// ACTIVATION FUNCTIONS
//


---


inline double sigmoid(double x)      { return 1.0 / (1.0 + exp(-x)); }
inline double sigmoid_deriv(double x) { double s = sigmoid(x); return s * (1.0 - s); }
inline double relu(double x)         { return max(0.0, x); }
inline double relu_deriv(double x)   { return x > 0.0 ? 1.0 : 0.0; }

//


---


// MLP CLASS
//


---


class MLP
{ public:
    int input_size, hidden_size, output_size;
    double lr;

    // Weights & biases
    vector<vector<double>>> W1, W2; // hidden & output weights
    vector<double>          b1, b2; // biases
```

```
// Cached activations
vector<double> z1, a1, z2, a2;
```

```
MLP(int in, int hid, int out, double learning_rate)
: input_size(in), hidden_size(hid), output_size(out), lr(learning_rate),
  W1(hid, vector<double>(in)), W2(out, vector<double>(hid)),
  b1(hid, 0.0), b2(out, 0.0),
  z1(hid), a1(hid), z2(out), a2(out) {
```

```
    mt19937 rng(123);
    normal_distribution<double> dist(0.0, 0.5);
    for (auto& row : W1) for (auto& w : row) w = dist(rng);
    for (auto& row : W2) for (auto& w : row) w = dist(rng);
}
```

```
// — Forward Pass (Parallelised) —————
```

```
void forward(const vector<double>& x) {
    // Layer 1:  $z1 = W1 \cdot x + b1$ ,  $a1 = \text{relu}(z1)$ 
    #pragma omp parallel for schedule(static)
    for (int j = 0; j < hidden_size; j++) {
        z1[j] = b1[j];
        for (int i = 0; i < input_size; i++)
            z1[j] += W1[j][i] * x[i];
        a1[j] = relu(z1[j]);
    }
    // Layer 2:  $z2 = W2 \cdot a1 + b2$ ,  $a2 = \text{sigmoid}(z2)$ 
    #pragma omp parallel for schedule(static)
    for (int k = 0; k < output_size; k++)
    {
        z2[k] = b2[k];
        for (int j = 0; j < hidden_size; j++)
            z2[k] += W2[k][j] * a1[j];
        a2[k] = sigmoid(z2[k]);
    }
}
```

```
// — Backpropagation (Parallelised)
```

```
————— void backward(const
vector<double>& x, const vector<double>& y) {
    // Output delta:  $\text{delta2} = (a2 - y) \cdot \text{sigmoid}'(z2)$ 
    vector<double> delta2(output_size);
    for (int k = 0; k < output_size; k++)
        delta2[k] = (a2[k] - y[k]) * sigmoid_deriv(z2[k]);

    // Hidden delta:  $\text{delta1} = (W2^T \cdot \text{delta2}) \cdot \text{relu}'(z1)$ 
```

```

vector<double> delta1(hidden_size, 0.0);
#pragma omp parallel for schedule(static)
for (int j = 0; j < hidden_size; j++) {
    for (int k = 0; k < output_size; k++)
        delta1[j] += W2[k][j] * delta2[k];
    delta1[j] *= relu_deriv(z1[j]);
}

// Update W2, b2 (parallel over output neurons)
#pragma omp parallel for schedule(static)
for (int k = 0; k < output_size; k++)
    { for (int j = 0; j < hidden_size; j+
      +)
        W2[k][j] -= lr * delta2[k] * a1[j];
      b2[k] -= lr * delta2[k];
    }

// Update W1, b1 (parallel over hidden neurons)
#pragma omp parallel for schedule(static)
for (int j = 0; j < hidden_size; j++)
    { for (int i = 0; i < input_size; i++)
        W1[j][i] -= lr * delta1[j] * x[i];
      b1[j] -= lr * delta1[j];
    }
}

// — MSE Loss ————— double
loss(const vector<double>& y_pred, const vector<double>& y_true) {
    double err = 0.0;
    for (int k = 0; k < output_size; k++)
        { double d = y_pred[k] - y_true[k];
          err += d * d;
        }
    return err / output_size;
}

// — Train One Epoch —————
double train_epoch(const vector<vector<double>>& X,
                  const vector<vector<double>>& Y)
{ double total_loss = 0.0;
  int n = X.size();
  for (int i = 0; i < n; i++)
      { forward(X[i]);
        backward(X[i], Y[i]);
        total_loss += loss(a2, Y[i]);
      }
}

```

```

    }
    return total_loss / n;
}

// — Predict
_____ int

predict(const vector<double>& x) {
    forward(x);
    return a2[0] >= 0.5 ? 1 : 0;
}
};

//
_____

// DATASET GENERATORS
//
_____

// XOR dataset (4 samples)
void make_xor(vector<vector<double>>& X, vector<vector<double>>& Y)
{
    X = {{0,0},{0,1},{1,0},{1,1}};
    Y = {{0}, {1}, {1}, {0}};
}

// Synthetic linearly-separable 2D dataset (n samples)
void make_circles(vector<vector<double>>& X, vector<vector<double>>& Y, int n)
{
    mt19937 rng(7);
    normal_distribution<double> noise(0, 0.15);
    X.resize(n, vector<double>(2));
    Y.resize(n, vector<double>(1));
    for (int i = 0; i < n; i++) {
        double angle = 2.0 * M_PI * i / n;
        int cls = (i % 2 == 0) ? 0 : 1;
        double r = cls == 0 ? 0.4 : 0.8;
        X[i][0] = r * cos(angle) + noise(rng);
        X[i][1] = r * sin(angle) + noise(rng);
        Y[i][0] = cls;
    }
}

//
_____

// PERFORMANCE BENCHMARK
//
_____

```

```

double train_and_time(MLP& net,
    const vector<vector<double>>& X,
    const vector<vector<double>>& Y,
    int epochs, bool verbose = true) {
    auto start = chrono::high_resolution_clock::now();
    for (int e = 1; e <= epochs; e++) {
        double l = net.train_epoch(X, Y);
        if (verbose && (e == 1 || e % (epochs/5) == 0))
            cout << " Epoch " << setw(4) << e << " Loss: " << fixed
                << setprecision(6) << l << endl;
    }
    auto end = chrono::high_resolution_clock::now();
    return chrono::duration<double, milli>(end-start).count();
}

//

// MAIN
//

int main() {
    cout << "===== " << endl;
    cout << " HPC for AI/ML: Parallel MLP Training (OpenMP) " << endl;
    cout << "===== " << endl;
    cout << "Threads available: " << omp_get_max_threads() << "\n\n";

    // — 1. XOR

    cout <<
    "--- Task 1: XOR Classification ---" << endl;
    vector<vector<double>> X_xor, Y_xor;
    make_xor(X_xor, Y_xor);

    MLP net_xor(2, 8, 1, 0.3);
    double t = train_and_time(net_xor, X_xor, Y_xor, 2000);
    cout << " Training time: " << t << " ms\n";
    cout << " Predictions:" << endl;
    string inputs[] = {"[0,0]", "[0,1]", "[1,0]", "[1,1]"};
    for (int i = 0; i < 4; i++)
        { net_xor.forward(X_xor[i])
          ;
          cout << " " << inputs[i] << " -> raw=" << fixed << setprecision(4)
              << net_xor.a2[0] << " pred=" << net_xor.predict(X_xor[i])
              << " true=" << (int)Y_xor[i][0] << endl;
        }
}

```

```

// — 2. Circles (large dataset)
----- cout << "\n--- Task 2:
Circles Classification (1000 samples) ---" << endl;
vector<vector<double>> X_circ, Y_circ;
make_circles(X_circ, Y_circ, 1000);

// Sequential (1 thread)
omp_set_num_threads(1);
MLP net_seq(2, 64, 1, 0.01);
cout << " [Sequential - 1 thread]" << endl;
double t_seq = train_and_time(net_seq, X_circ, Y_circ, 100, true);
cout << " Training time: " << t_seq << " ms\n";

// Parallel (all threads)
omp_set_num_threads(omp_get_max_threads());
MLP net_par(2, 64, 1, 0.01);
cout << "\n [Parallel - " << omp_get_max_threads() << " threads]" << endl;
double t_par = train_and_time(net_par, X_circ, Y_circ, 100, true);
cout << " Training time: " << t_par << " ms\n";
cout << " Speedup:      " << t_seq/t_par << "x\n";

// — 3. Accuracy
----- int correct = 0;
for (int i = 0; i < (int)X_circ.size(); i++)
    if (net_par.predict(X_circ[i]) == (int)Y_circ[i][0]) correct++;
cout << "\n Final Accuracy: " << fixed << setprecision(2)
    << 100.0 * correct / X_circ.size() << "%" << endl;

cout << "\n===== " << endl;
cout << " Summary: OpenMP parallelises forward/backward " << endl;
cout << " passes across neurons enabling significant " << endl;
cout << " speedup for large hidden layers. " << endl;
cout << "===== " << endl;
return 0;
}

// — Compile & Run -----
// g++ -fopenmp -O2 -o a5 assignment5_hpc_ml.cpp && ./a5

```

Output:

```
=====
Threads available: 5
```

```
--- Task 1: XOR Classification ---
```

```
Epoch   1  Loss: 0.292878
```

```
Epoch  400  Loss: 0.002493
```

```
Epoch  800  Loss: 0.000843
```

```
Epoch 1200  Loss: 0.000478
```

```
Epoch 1600  Loss: 0.000326
```

```
Epoch 2000  Loss: 0.000244
```

```
Training time: 210.412557 ms
```

```
Predictions:
```

```
[0,0] -> raw=0.0186  pred=0  true=0
```

```
[0,1] -> raw=0.9833  pred=1  true=1
```

```
[1,0] -> raw=0.9874  pred=1  true=1
```

```
[1,1] -> raw=0.0137  pred=0  true=0
```

```
--- Task 2: Circles Classification (1000 samples) ---
```

```
[Sequential - 1 thread]
```

```
Epoch   1  Loss: 0.240345
```

```
Epoch  20  Loss: 0.085082
```

```
Epoch  40  Loss: 0.078318
```

```
Epoch  60  Loss: 0.076132
```

```
Epoch  80  Loss: 0.075087
```

```
Epoch 100  Loss: 0.074516
```

```
Training time: 410.389460 ms
```

```
[Parallel - 1 threads]
```

```
Epoch   1  Loss: 0.240345
```

```
Epoch  20  Loss: 0.085082
```

```
Epoch  40  Loss: 0.078318
```

```
Epoch  60  Loss: 0.076132
```

```
Epoch  80  Loss: 0.075087
```

```
Epoch 100  Loss: 0.074516
```

```
Training time: 378.890102 ms
```

```
Speedup:      1.083136x
```

```
Final Accuracy: 89.30%
```

```
=====
Summary: OpenMP parallelises forward/backward
passes across neurons enabling significant
speedup for large hidden layers.
=====
```

```
vboxuser@ubuntu:~/LP V_Code/HPC$ █
```